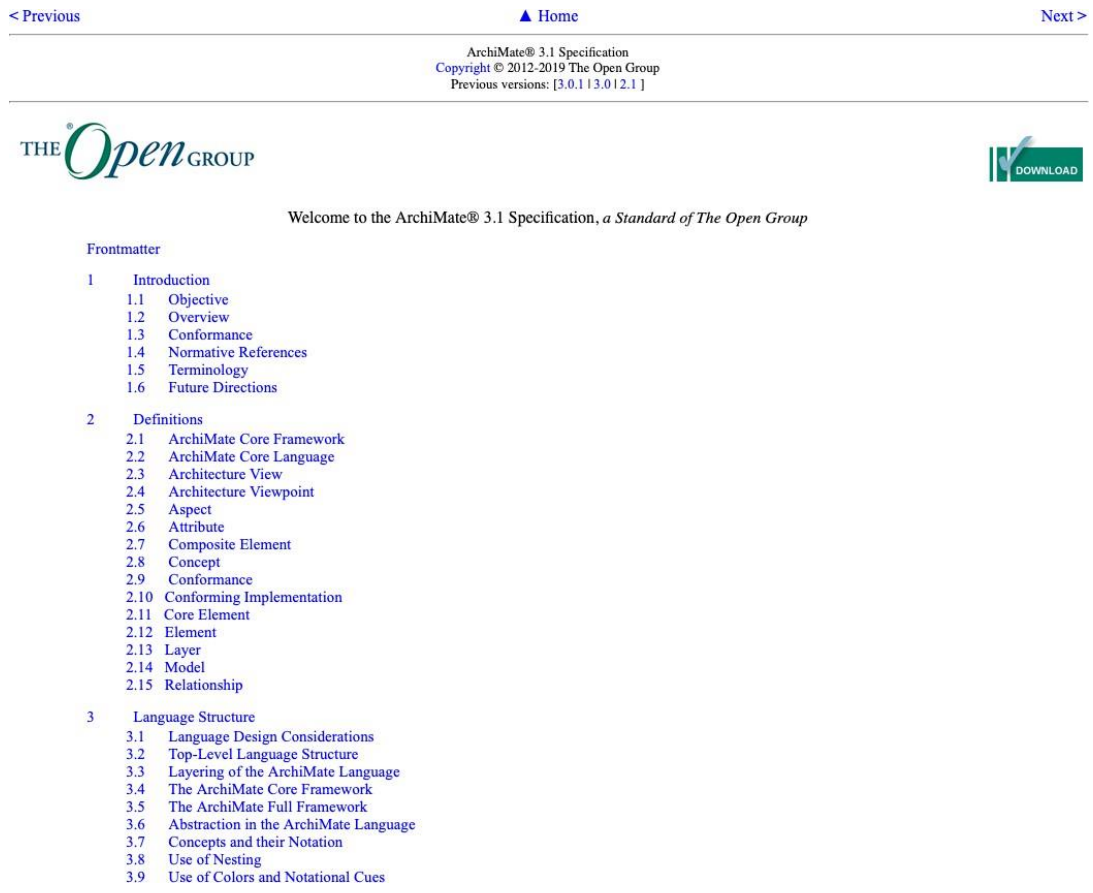
ArchiMate
Application Layer

Specifica ArchiMate

https://pubs.opengroup.org/architecture/archimate3-doc/ch-Application-Layer.html



ArchiMate® 3.1 Specification
Copyright © 2012-2019 The Open Group
Previous versions: [3.0.1 | 3.0 | 2.1]

THE Open GROUP

DOWNLOAD

Welcome to the ArchiMate® 3.1 Specification, *a Standard of The Open Group*

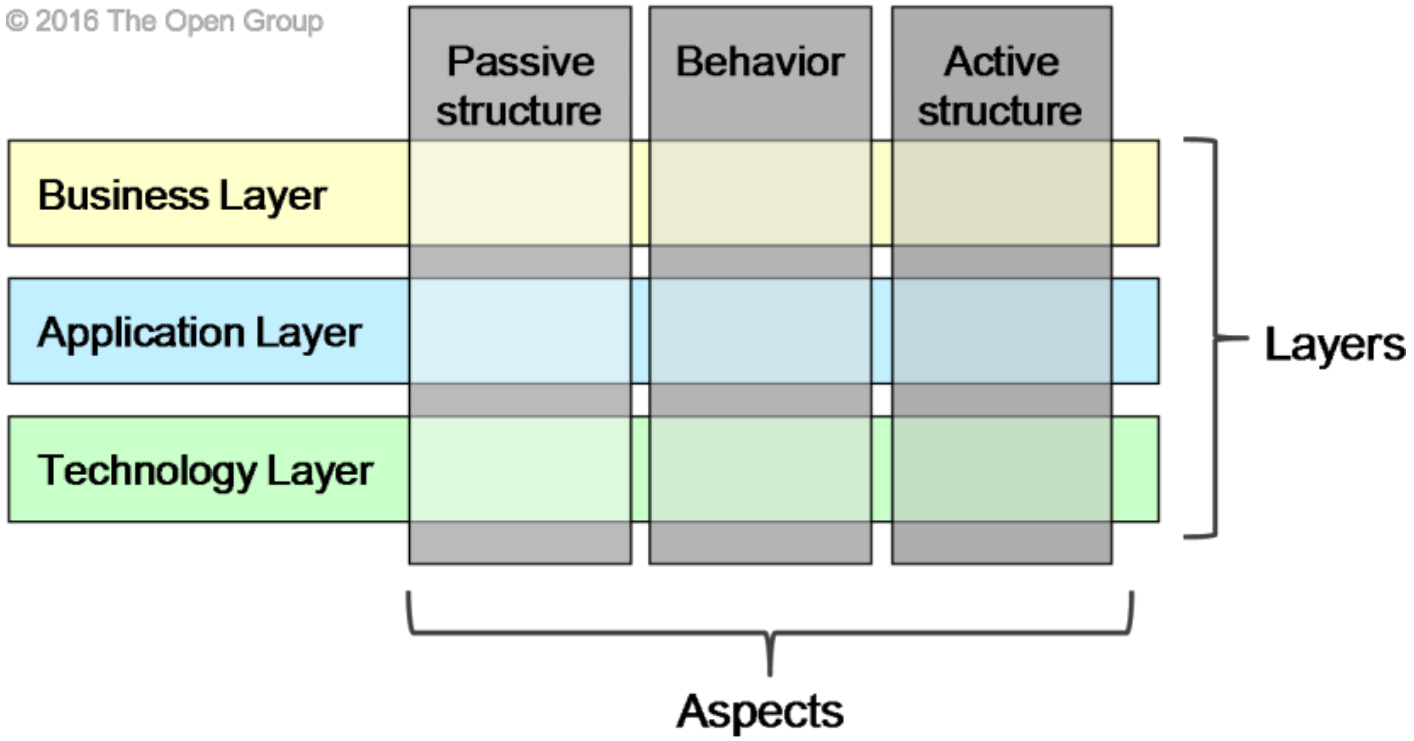
Frontmatter

1 Introduction
- 1.1 Objective
- 1.2 Overview
- 1.3 Conformance
- 1.4 Normative References
- 1.5 Terminology
- 1.6 Future Directions

2 Definitions
- 2.1 ArchiMate Core Framework
- 2.2 ArchiMate Core Language
- 2.3 Architecture View
- 2.4 Architecture Viewpoint
- 2.5 Aspect
- 2.6 Attribute
- 2.7 Composite Element
- 2.8 Concept
- 2.9 Conformance
- 2.10 Conforming Implementation
- 2.11 Core Element
- 2.12 Element
- 2.13 Layer
- 2.14 Model
- 2.15 Relationship

3 Language Structure
- 3.1 Language Design Considerations
- 3.2 Top-Level Language Structure
- 3.3 Layering of the ArchiMate Language
- 3.4 The ArchiMate Core Framework
- 3.5 The ArchiMate Full Framework
- 3.6 Abstraction in the ArchiMate Language
- 3.7 Concepts and their Notation
- 3.8 Use of Nesting
- 3.9 Use of Colors and Notational Cues

ArchiMate CORE framework



ArchiMate Core describe the different layers/domains of the Enterprise Architecture:

Business Layer: Goals and Requirements
Application Layer: system functionality
Infrastructural Layer: technological choices (data exchange platforms, security...)

Application Layer

The Application Layer in ArchiMate describes the structure, behavior, and interactions of software applications within an organization that supports the business.

It models application components, their internal behavior, the services they provide towards the business.

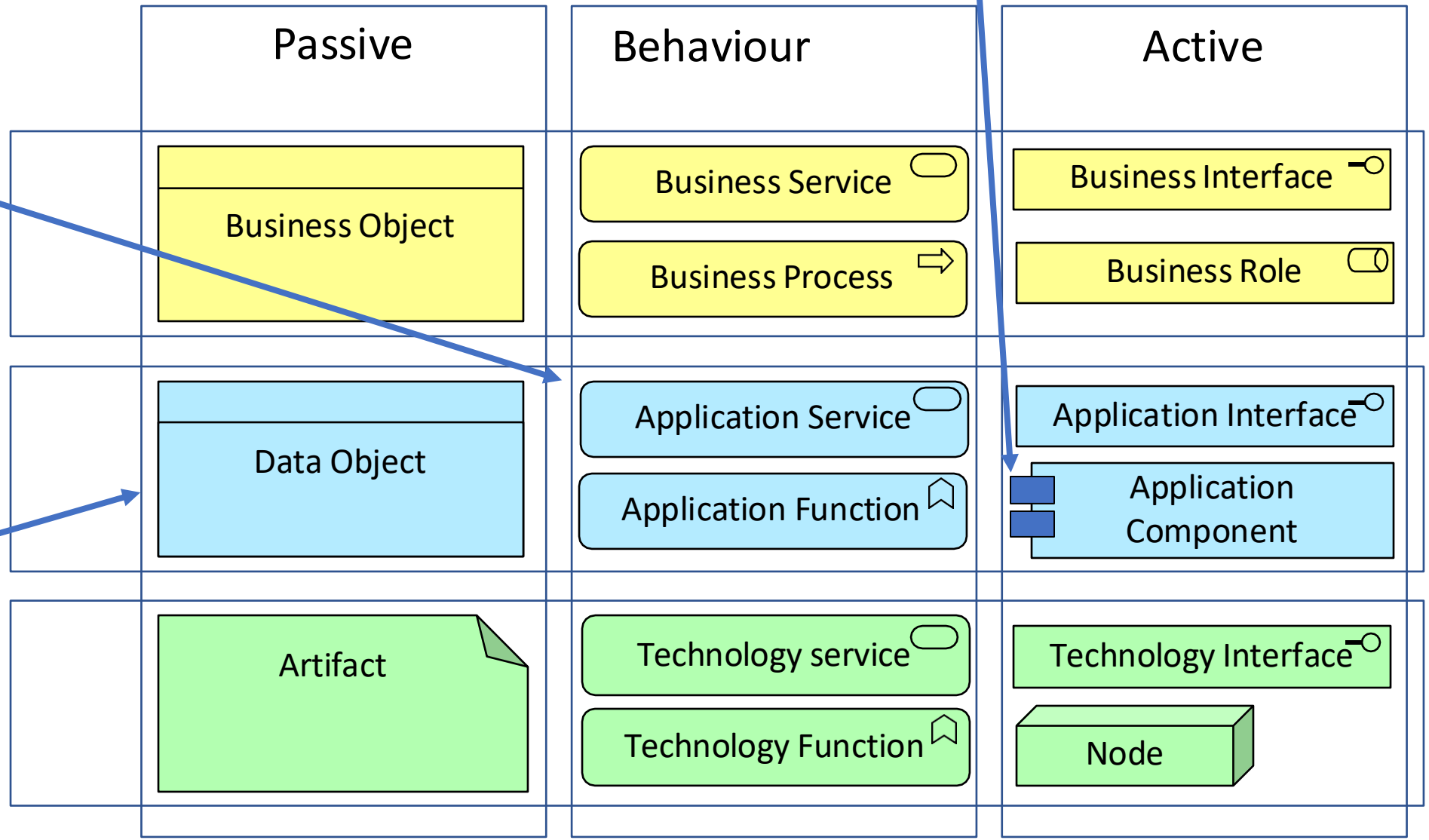
Key questions: How do applications support and realize business requirements and processes? Which applications deliver which services to support the business?

Application Layer

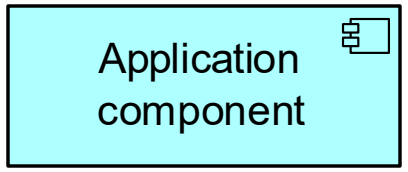
Active application components perform an application/task

What is executed automatically and exposed?

Data exchanged (e.g. relationships in the DB, objects)



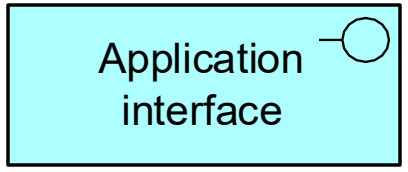
Active elements



Module that guarantees the offering/execution of a certain functionality

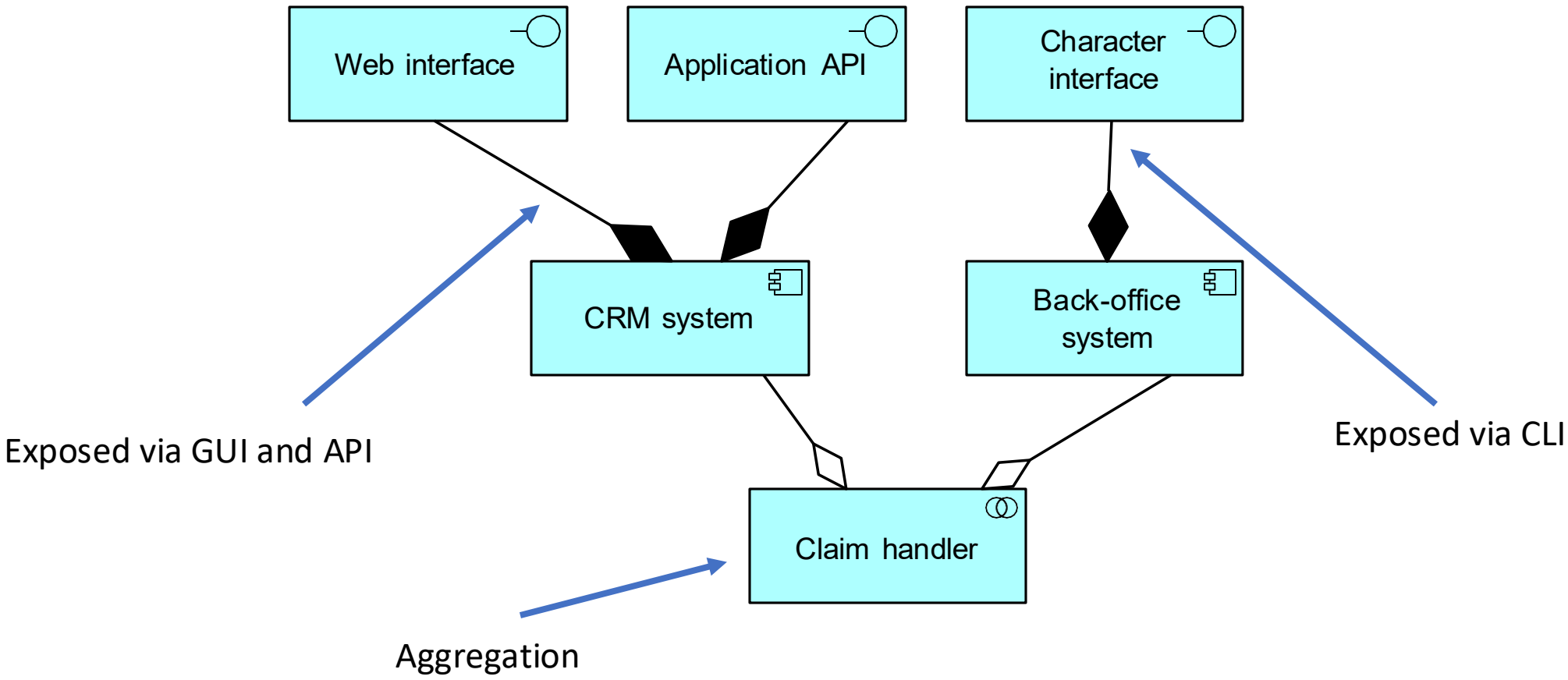


Aggregation of one or more application component modules for composing a collective behavior



How a component can be accessed. At the application level a module can be offered via GUI / CLI / API

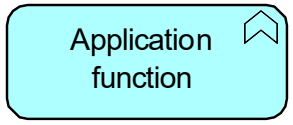
Active Element Relationships



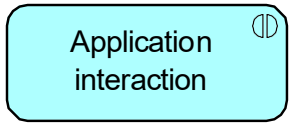
Behavioral elements



A sequence of application behaviors that achieves a specific purpose. Example: Process Customer Order / Verify Login



Aggregation of behavioral elements that can be performed by an application component that exposes a given functionality. Example Customer Data Managment / Authentication



A collective application that is run by one or more Application Components (Collaboration)

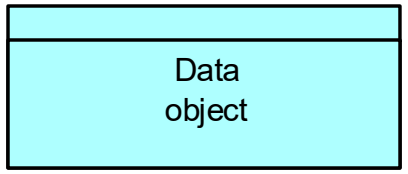


Application service: An explicitly defined application behavior is how functionality is exposed, through an appropriate interface



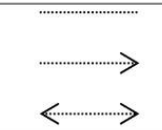
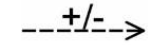
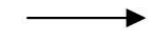
Application event: a behavioral element that denotes a change in state

Passive elements



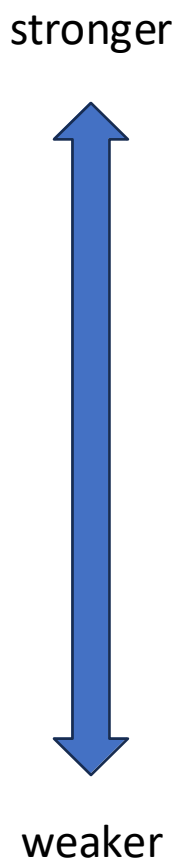
Element that allows to characterize structured information / data for information processing

Relationships

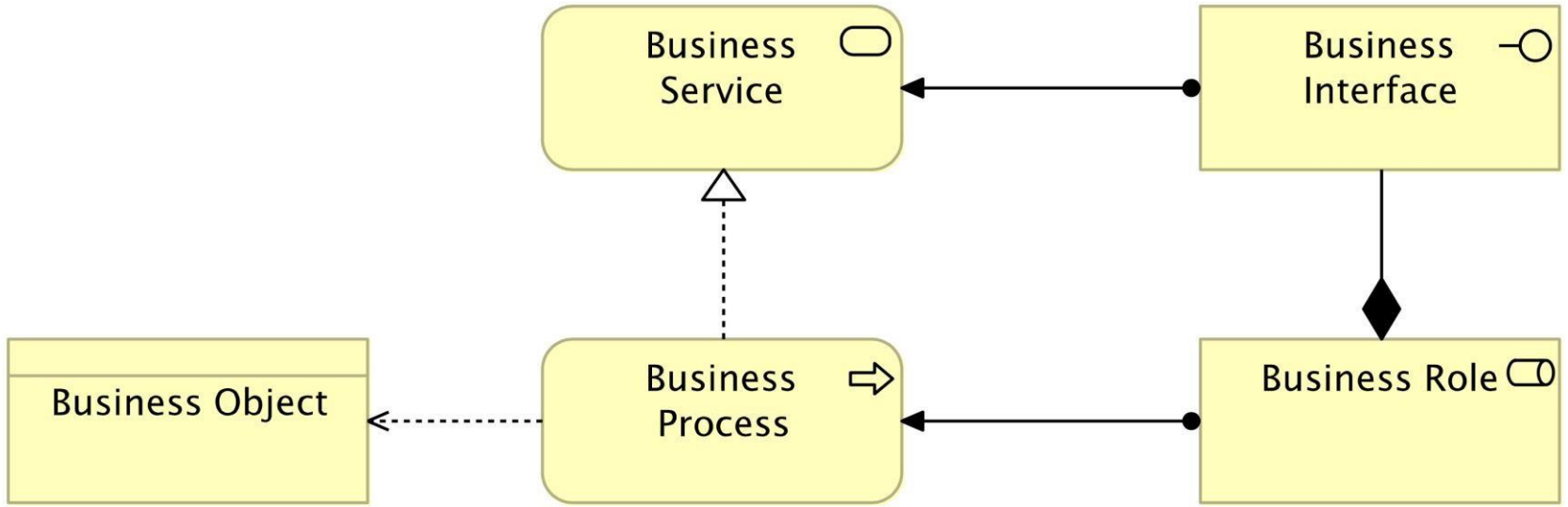
Structural Relationships		**Notation**
Composition	Indicates that an element consists of one or more other concepts.	
Aggregation	indicates that an element groups a number of other concepts.	
Assignment	Expresses the allocation of responsibility, performance of behavior, or execution.	
Realization	Indicates that an entity plays a critical role in the creation, achievement, sustenance, or operation of a more abstract entity.	
Dependency Relationships		**Notation**
Serving	Models that an element provides its functionality to another element. "Used by"	
Access	Models the ability of behavior and active structure elements to observe or act upon passive structure elements.	
Influence	Models that an element affects the implementation or achievement of some motivation element.	
Dynamic Relationships		**Notation**
Triggering	Describes a temporal or causal relationship between elements.	
Flow	Transfer from one element to another.	
Other Relationships		**Notation**
Specialization	Indicates that an element is a particular kind of another element.	
Association	Models an unspecified relationship, or one that is not represented by another ArchiMate relationship.	
Junction	Used to connect relationships of the same type.	

Strengths of relationships

Structural R	**Notation**
Composition	
Aggregation	
Assignment	
Realization	
Dependency	**Notation**
Serving	
Access	
Influence	+/-
Dynamic Rel	**Notation**
Triggering	
Flow	
Other Relati	**Notation**
Specialization	
Association	



Pattern base (business layer)



This is the Access relation

This is the Composition relation

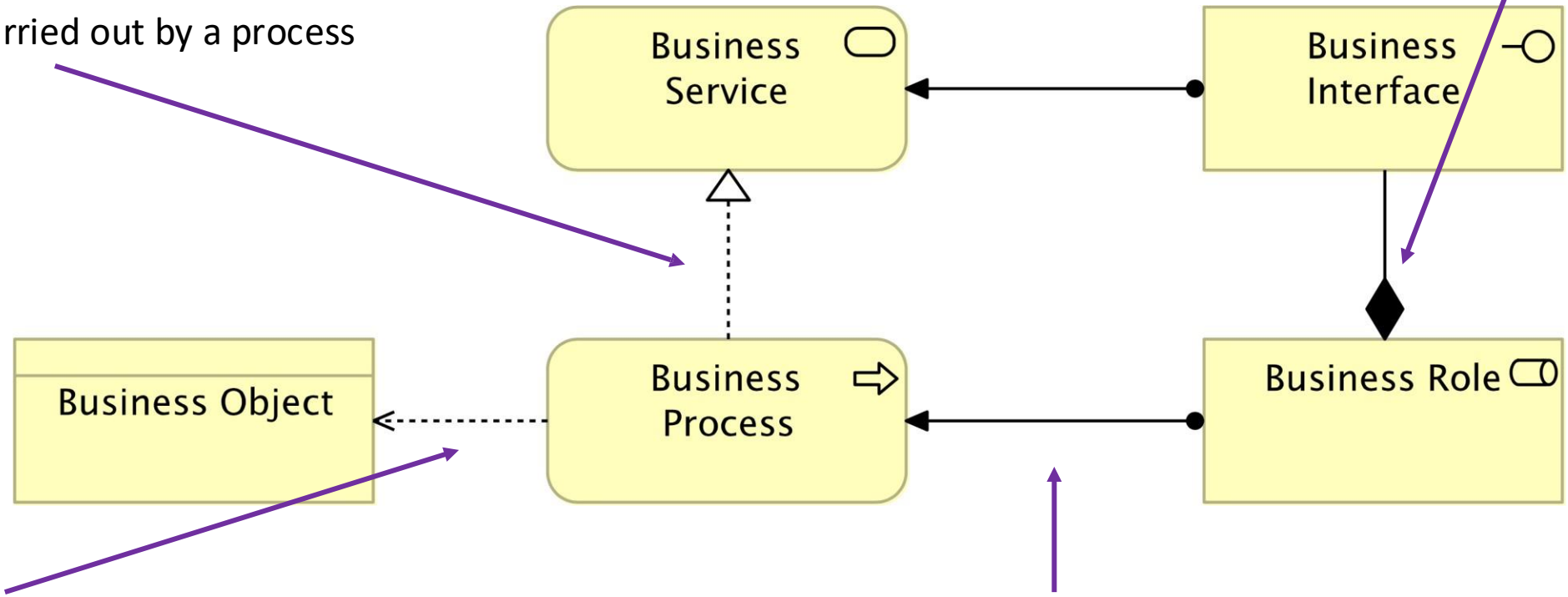
This is the Realization relation

This is the Assignment relation

Pattern base (business layer)

Roles make up an interface to the customer

The service is carried out by a process

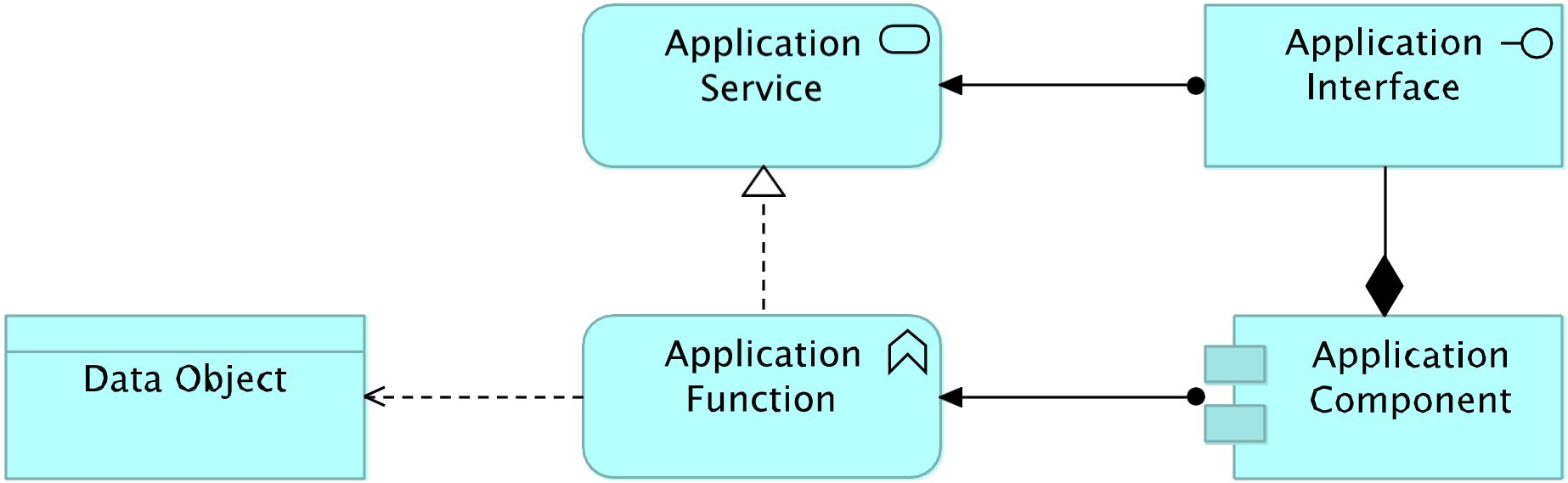


The process uses objects as inputs and produces outputs

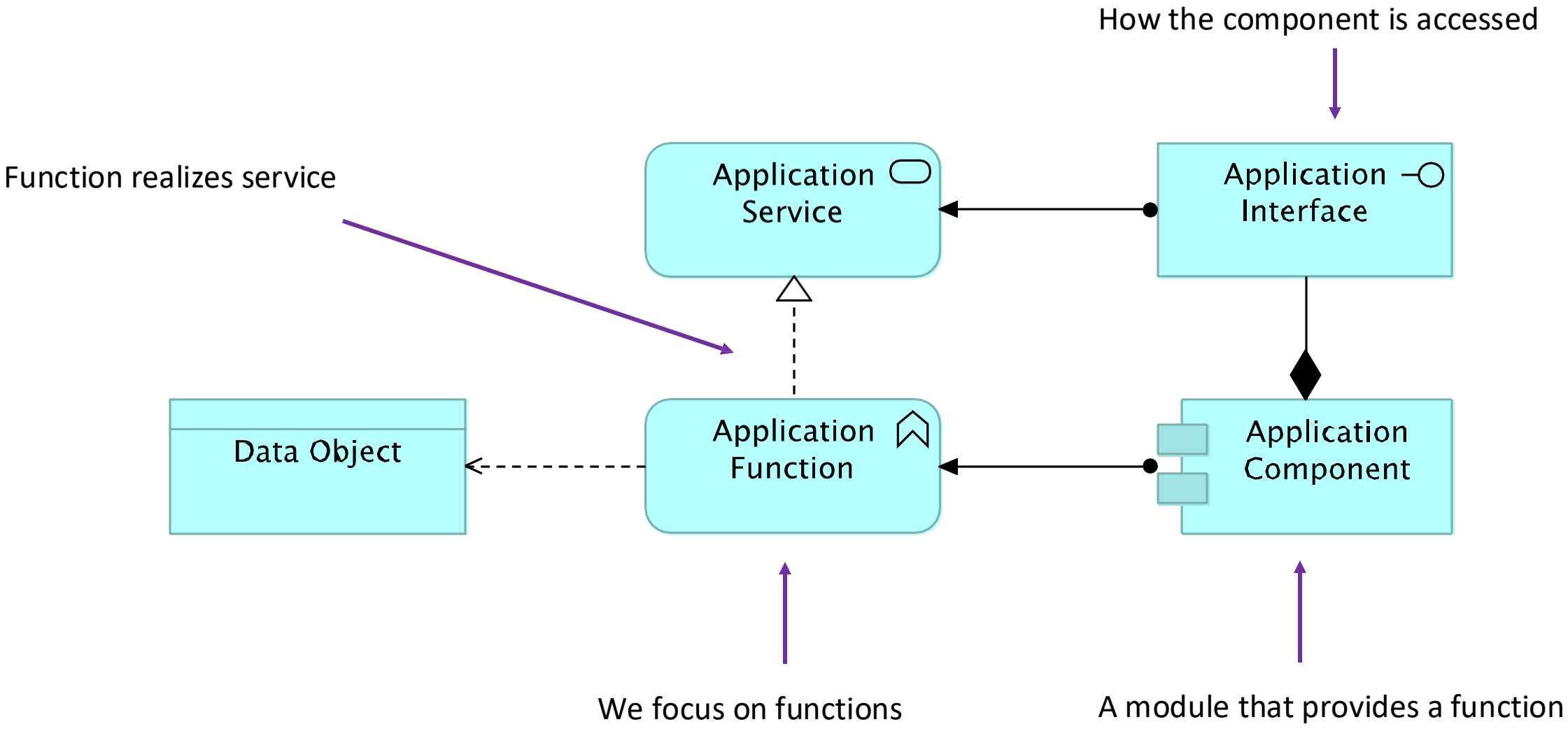
The role is assigned to a process

I use the composition between role and interface because if I delete a role I plausibly delete the interface as well. Instead, service and process have a weaker relationship: I can offer the same service by changing the process

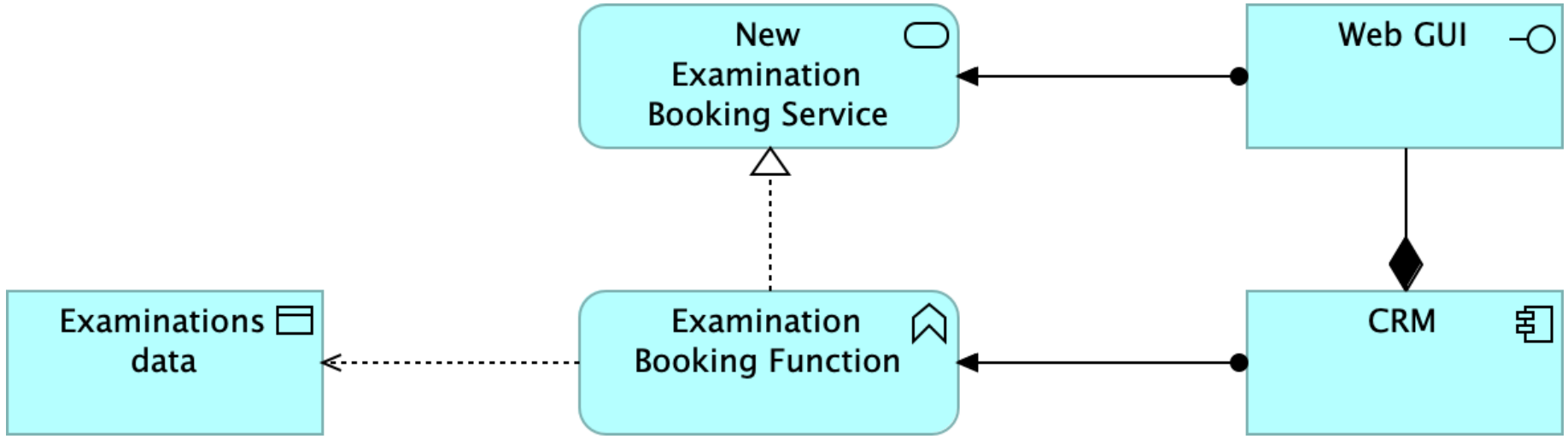
Pattern base (application layer)



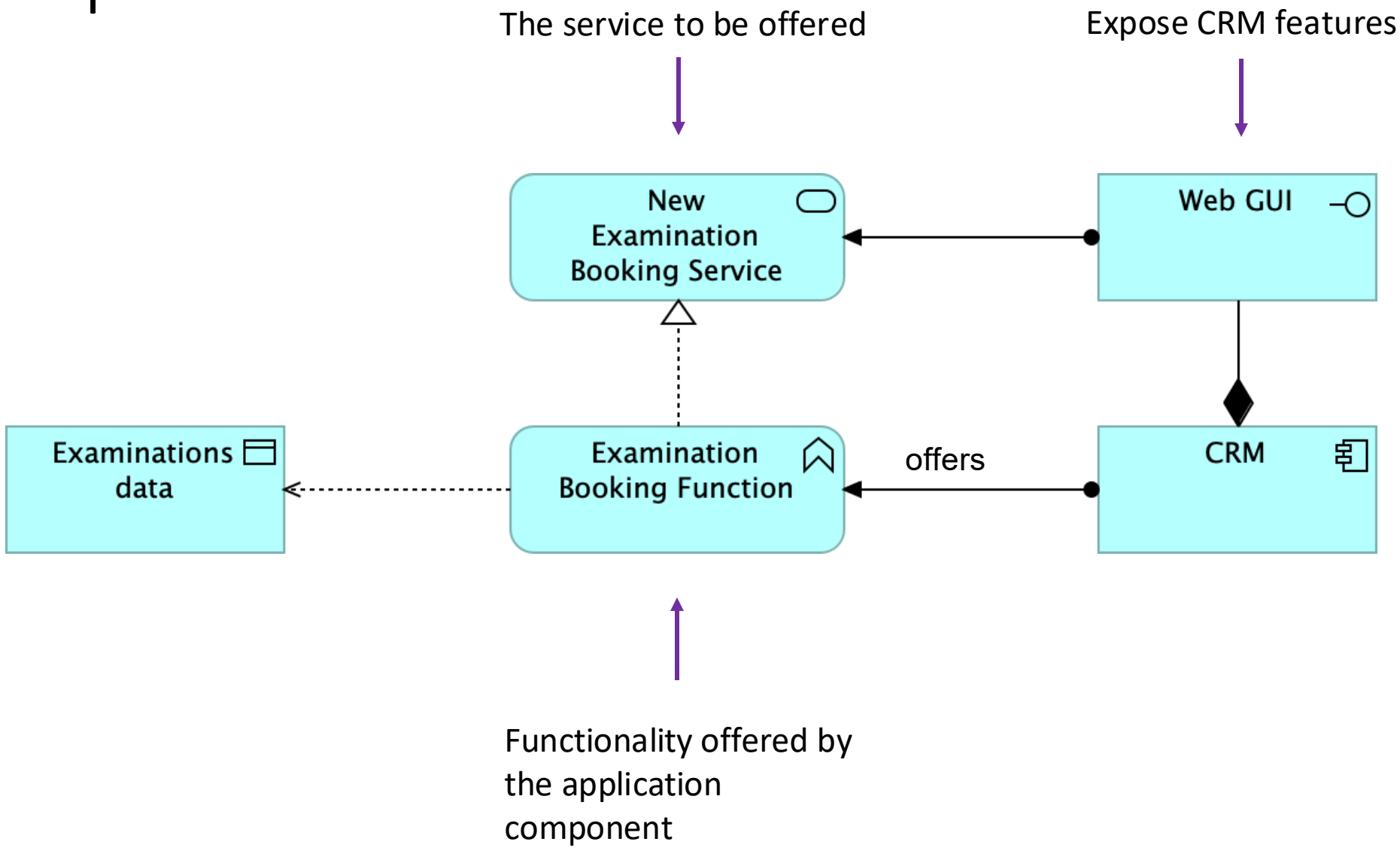
Pattern base (application layer)



Example


New
Examination
Booking Service
Web GUI
Examinations
data
Examination
Booking Function
CRM

Example


The service to be offered
Expose CRM features
New Examination Booking Service
Web GUI
Examinations data
Examination Booking Function
offers
CRM
Functionality offered by the application component

Structure and behavior

ArchiMate at the application level distinguishes between

- Structural elements → *components*

  They provide the features

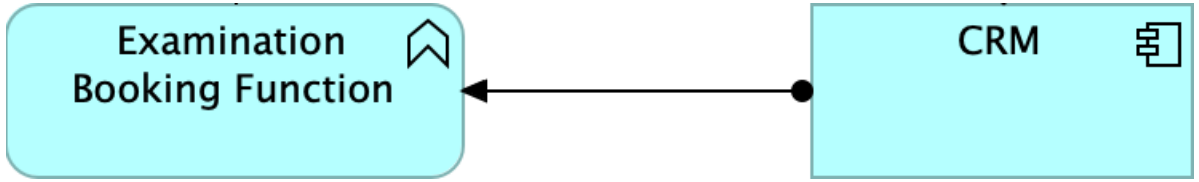
- Behavioral elements → *functions*

  What is the feature? How does the feature work?

In the application layer, it can be difficult to distinguish between

Who executes (the active element, i.e., the component)

What is performed (the behavioral element, i.e., the function or the process)



Functions and processes (application layer)

Functions and **processes** are performed by a single component

If multiple components are involved, a junction or collaboration is used to make it explicit

Functions and processes are aggregations are composed of other behavioral elements

Functions when:

Inner Elements have something in common (such as role or resources)

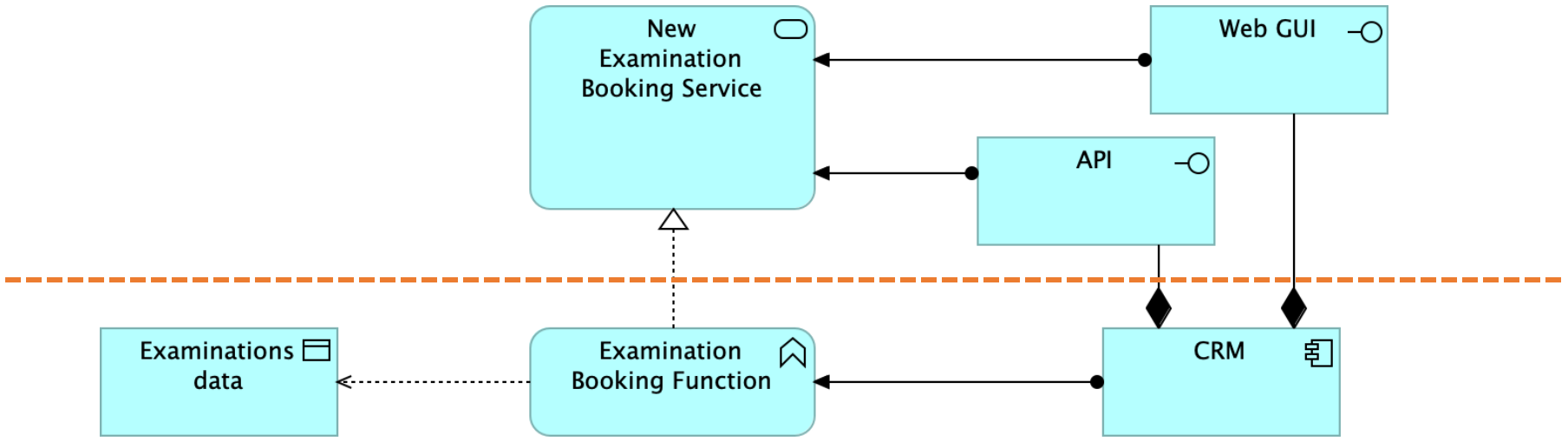
Processes when:

Inner elements collectively achieve a certain purpose (with a control flow)

For simplicity, in the **application layer** we will only use **functions**

Above visibility line

- Business processes or functions "see" an accessible service through a given interface
- The interfaces can normally be more than one (even without the use of junctions)

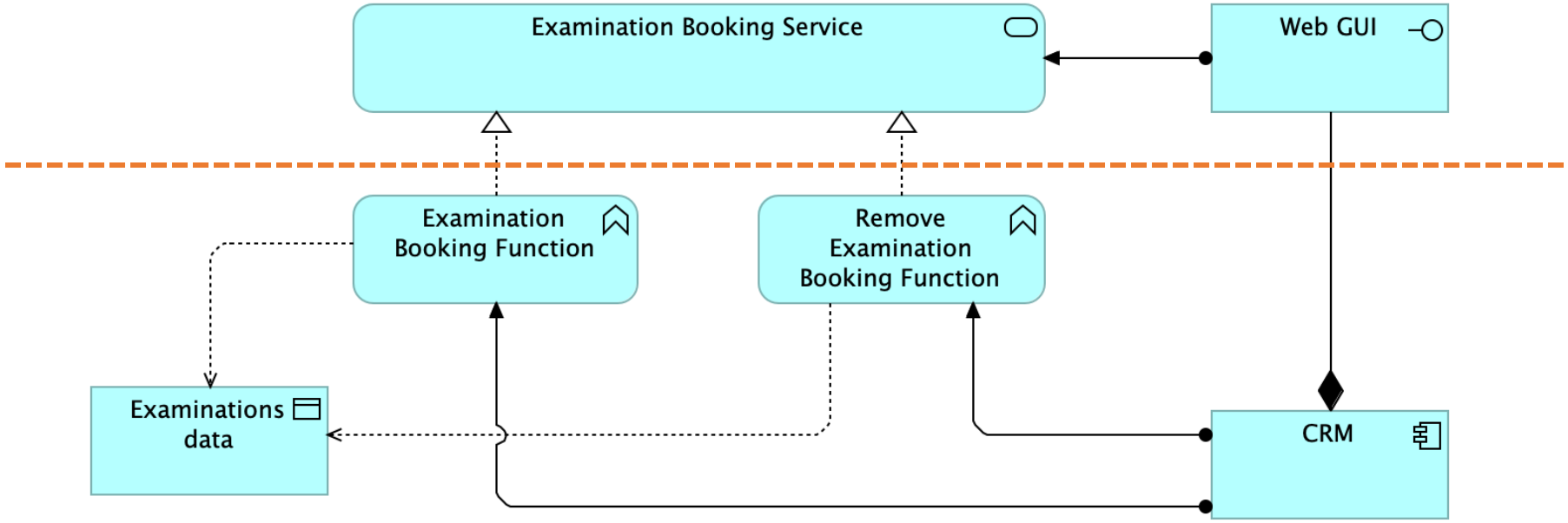


Behind Visibility Line

- How is the service offered, in terms of features and modules that can offer it

Example. CRM offers two functions that produce the same service

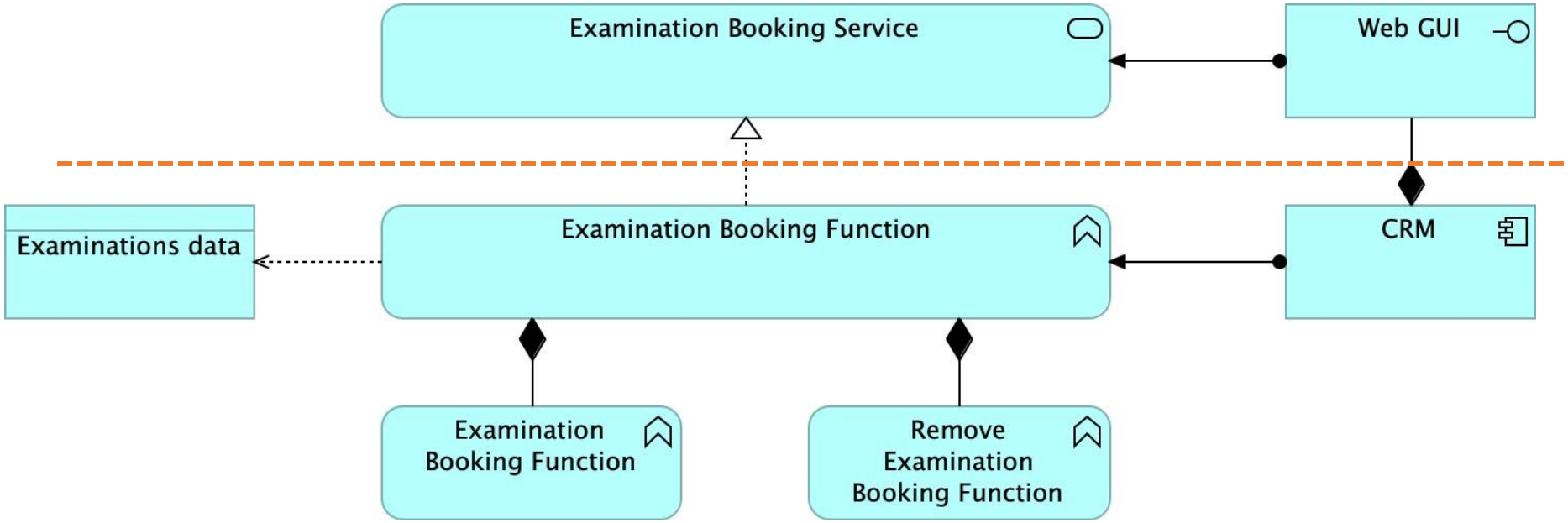
(Functions are not visible to the »user»)



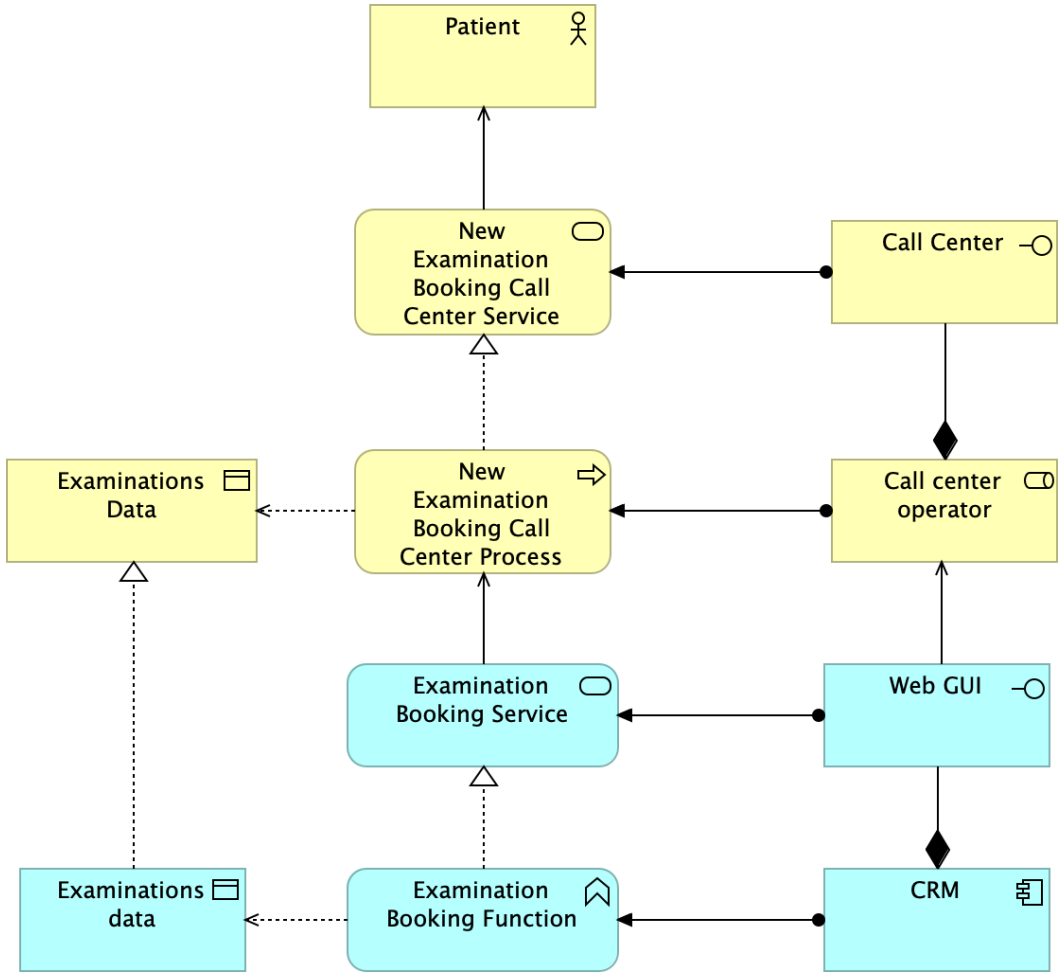
Behind Visibility Line

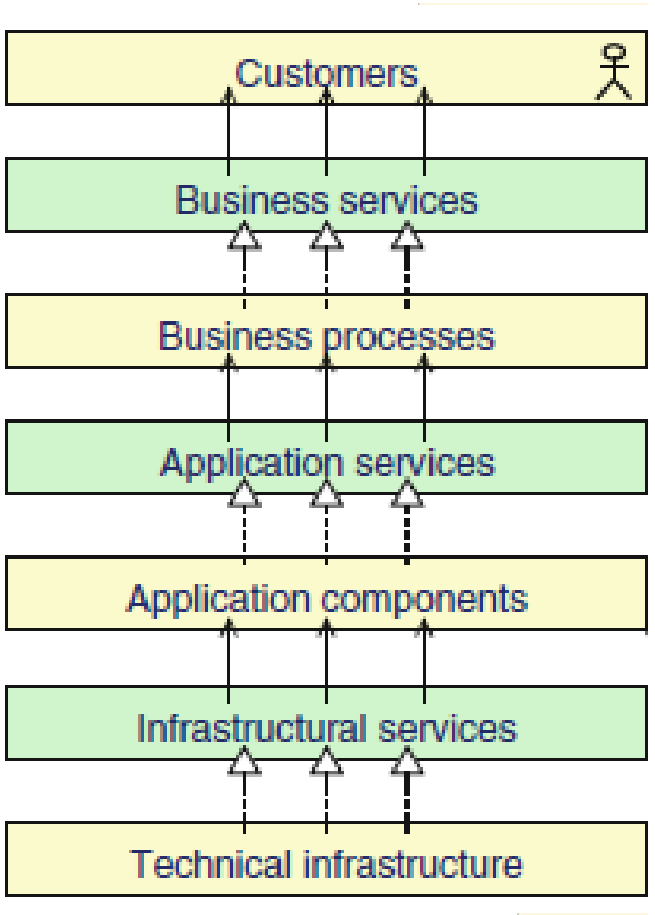
- You can use composition to aggregate functions and make the model more compact

Do you remember how we did it in previous lesson?



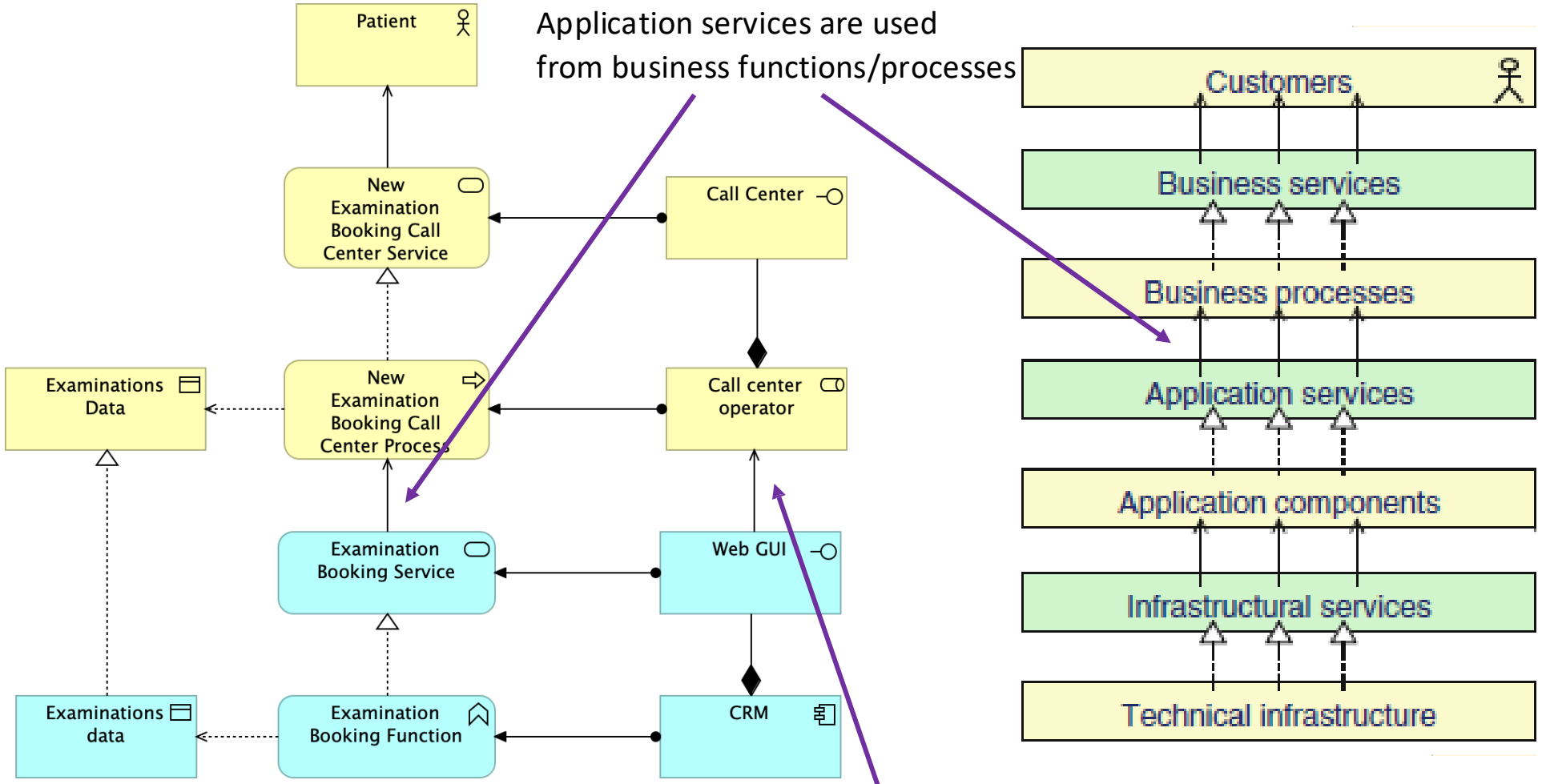
Application & Business Layer Connection





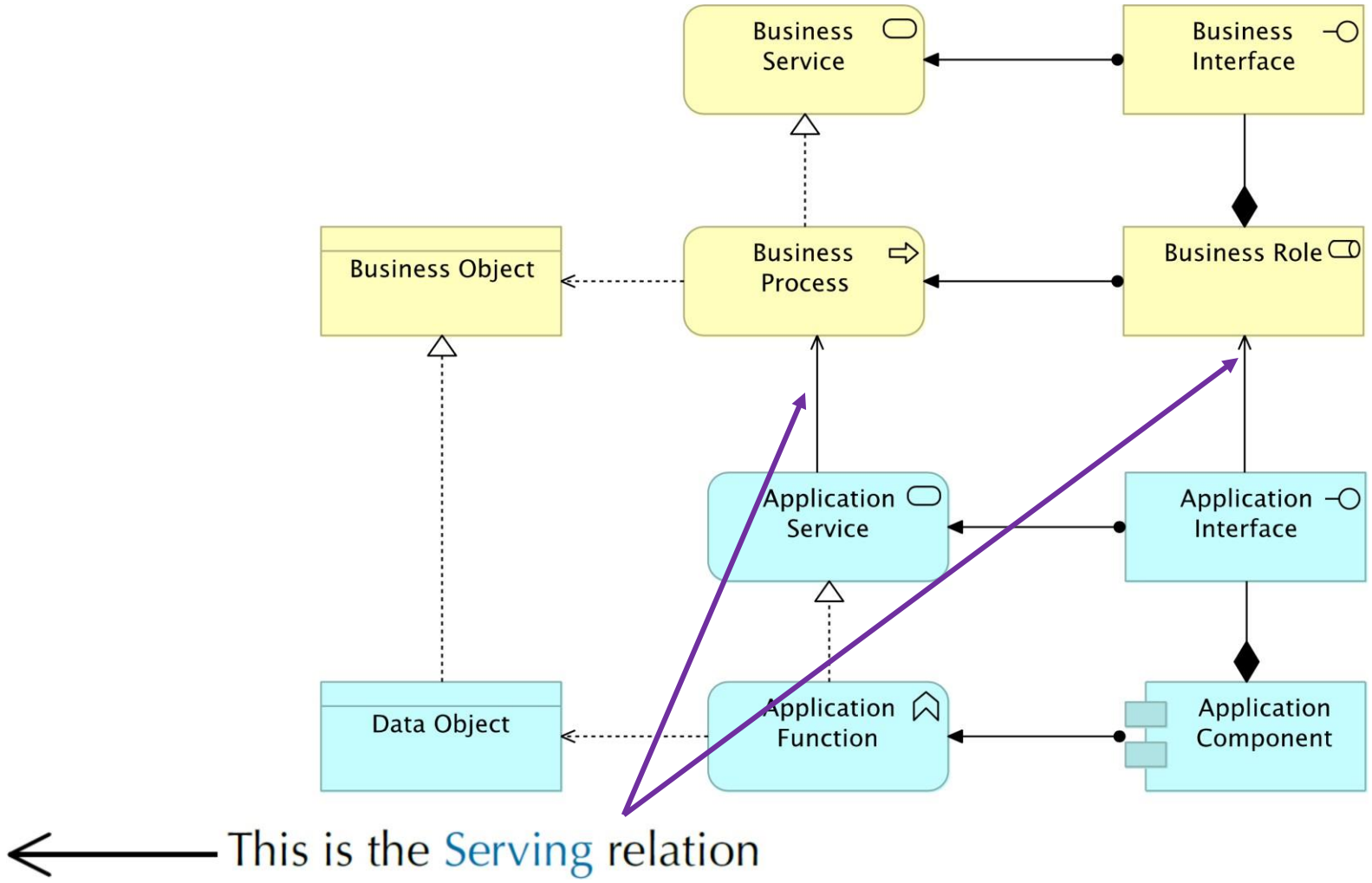
Application & Business Layer Connection

A visible element of the pattern connects with an implementation element of the top layer



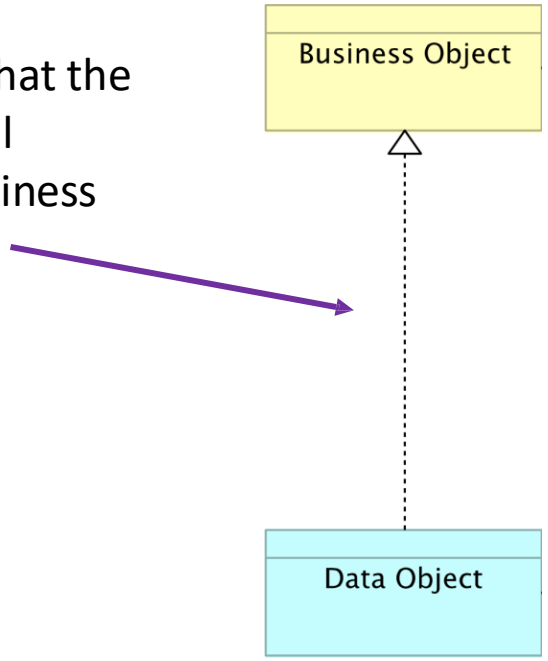
The application module interface is used by the role that controls the execution of the business layer (NB: they must be compatible)

Pattern – Business + Application



Pattern – Business + Application

"Realize" in the sense that the data object is the digital representation of a business Object

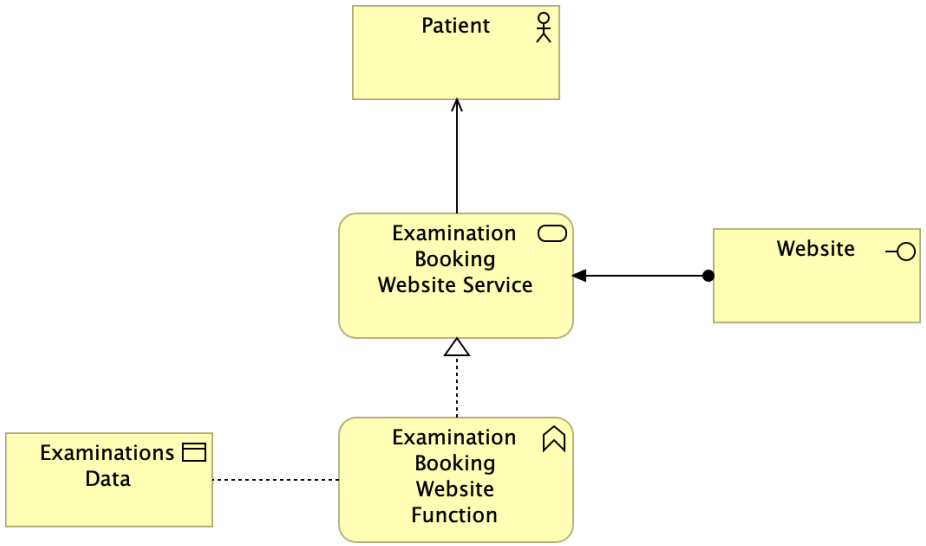


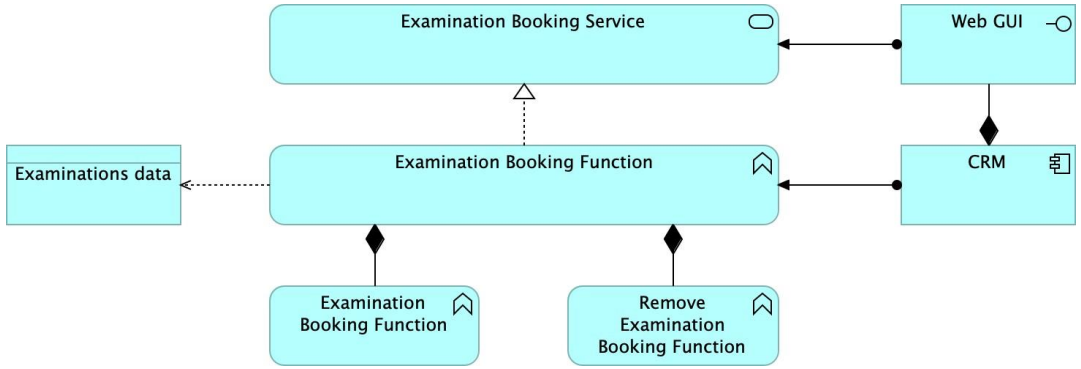
Business objects represent **knowledge or information used** and manipulated in the business layer, while **data objects** represent **data that is stored, processed, and managed** in the application layer to create business objects

Examples:
Customer Profile -> Customer Record
Invoice -> Invoice data table

A self-serving service

At the application level, nothing changes, but there is no longer a role that accesses the application interface





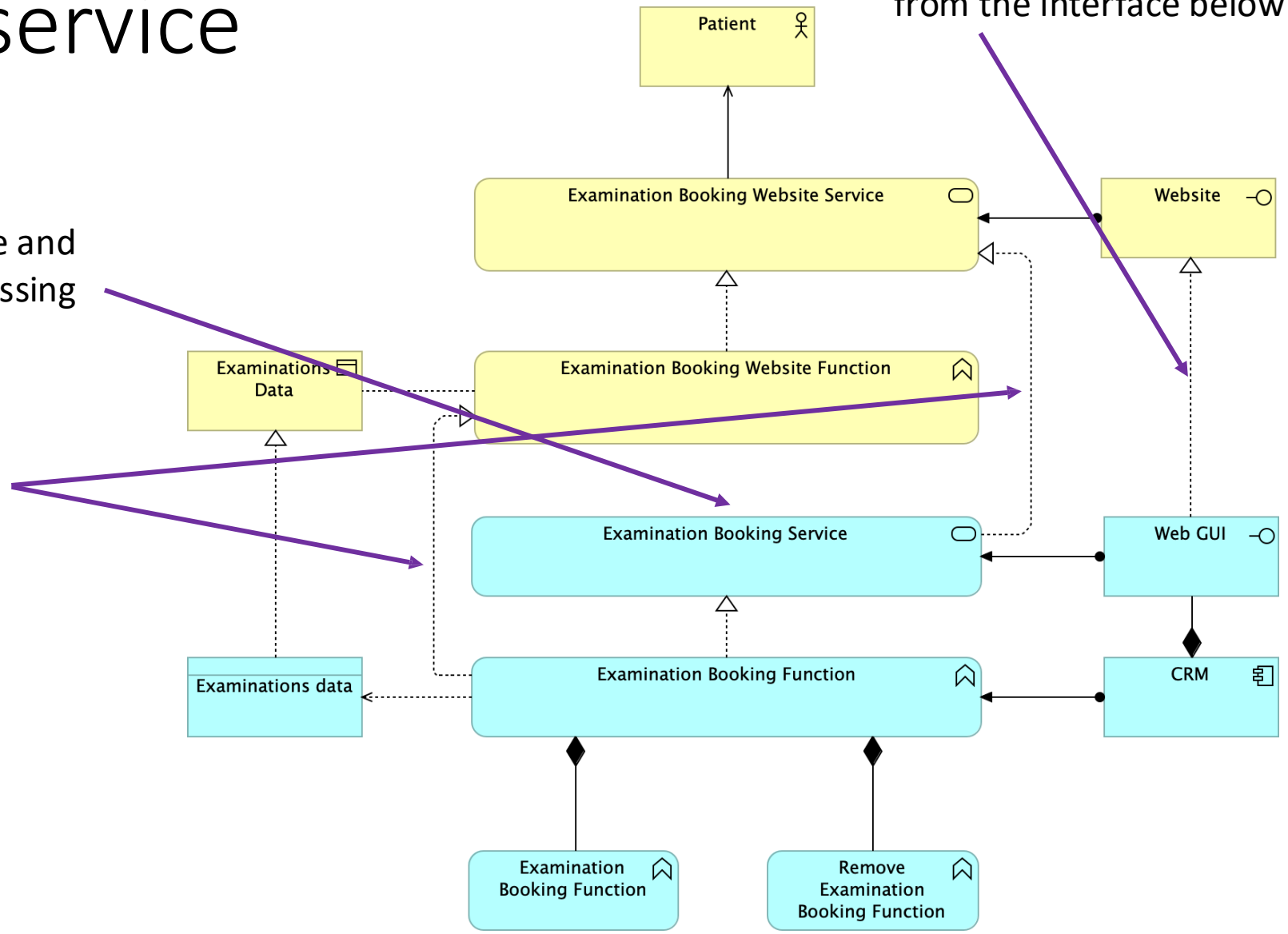
But we need to link them.. How?

A self-serving service

The website is created from the interface below

The link between application service and business layer implementation is missing

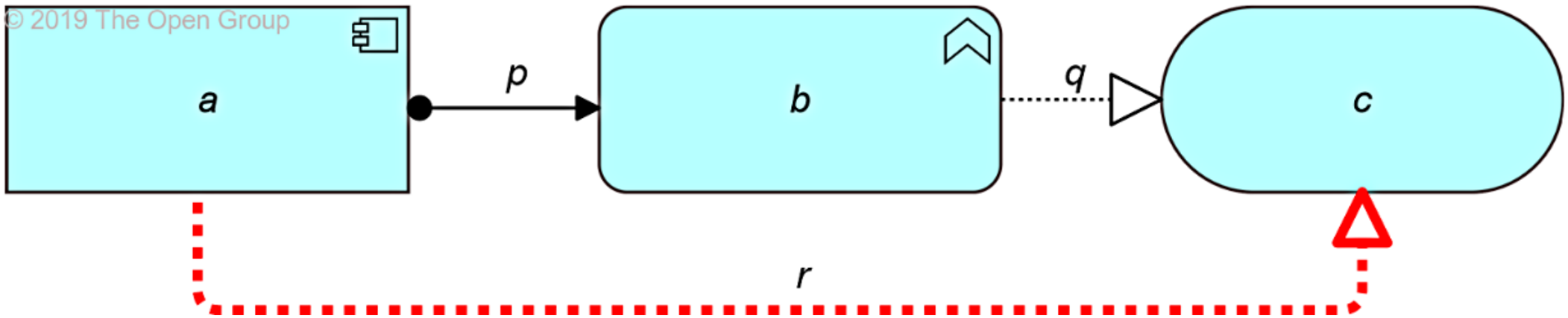
The application function realizes the business function (there are no "intermediaries" of the functionalities, the service application is "seen" directly by the end user). There is no Business Role.. That's the reason why



Derivative Relationships

The rule for structural and dependency relationships:

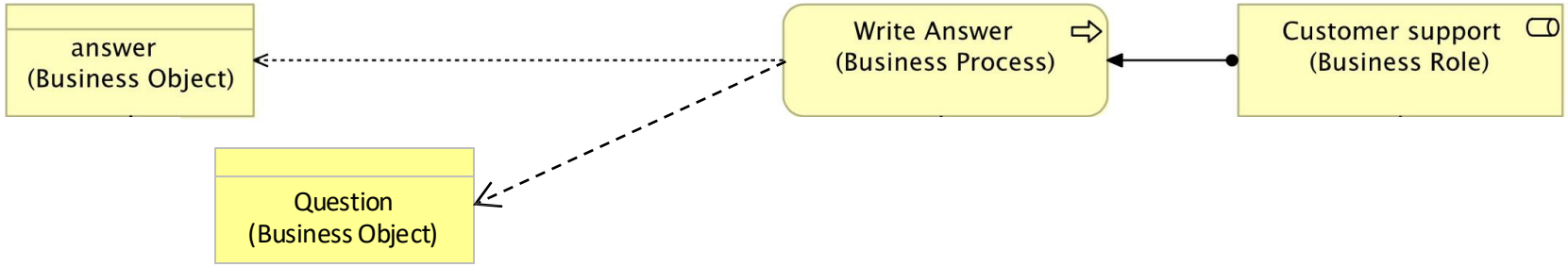
1. *If two relationships p(a,b):S and q(b,c):T exist, with S and T being structural relationships, then a relationship r(a,c):U can be derived, with U being the weakest of S and T.*
2. *If two relationships p(a,b):S and q(b,c):T exist, with S being a structural relationship and T being a dependency relationship, then a relationship r(a,c):T can be derived.*



ArchiMate®
Exercises

Exercise – Client Answer

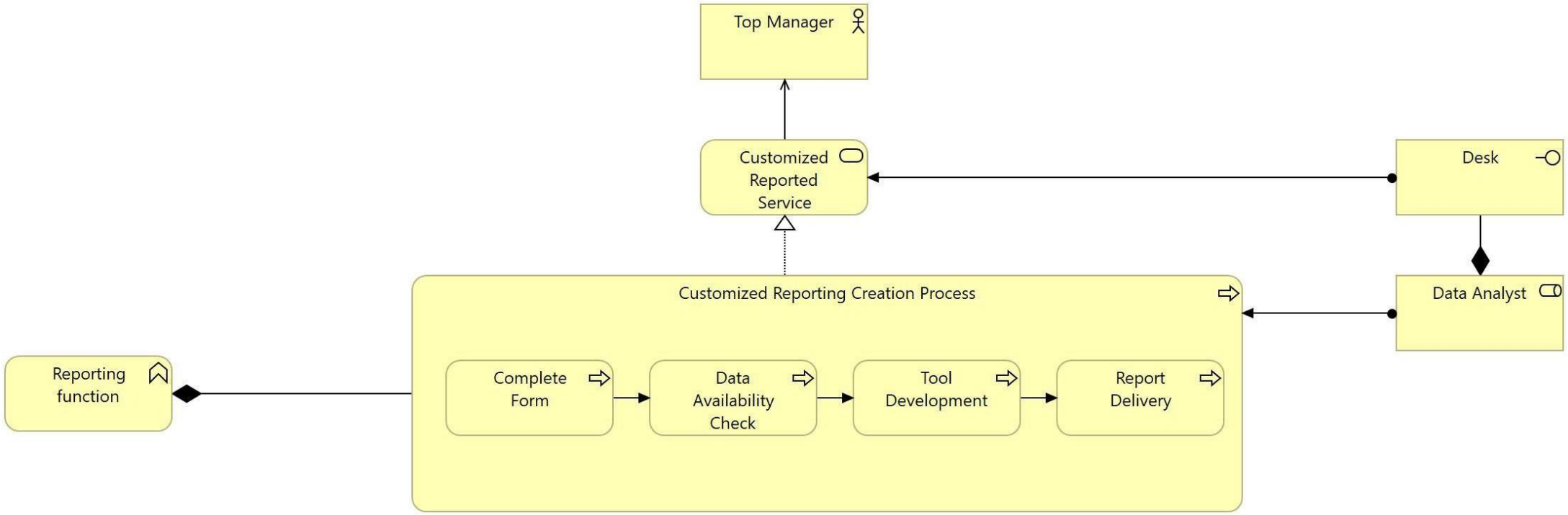
Use an ArchiMate model to describe the Enterprise Architecture needed for a small company's customer service, regarding the need to answer customer questions.


answer
(Business Object)
Write Answer
(Business Process)
Customer support
(Business Role)
Question
(Business Object)

- Speedy 01 – Business Layer

Speedy is a delivery company that wants to offer to its top management a new service to create custom reports. The company already offers some services in this direction but the produced reports are not customizable. For this new service, a data analyst is available to the managers to understand their requirements and to develop the new report.

To achieve this, the data analyst will require you to fill out a form, verify the available data, develop the tools needed to produce the report, and finally deliver that report to the manager.



Archi

- Speedy 01 - Application layer

To offer this service, Speedy relies on a data warehouse accessible through a web interface.
Specifically, a reporting module, which is offered by the data warehouse, is in charge of interacting with the managed data marts to enable the activities which can be performed by the user (data analyst). To this aim, the data warehouse periodically reads the data about the products from the organization's ERP system through an APl, to create the data mart that is used by the reporting module.

- IC 01 – Business Layer

IC is an insurance company which wants to offer a new insurance service for small objects (< 2000$) managed completely online for reliable customers. The new process starts when new insurance request arrives.

Then the customers credentials and past history are checked. If the customer is considered reliable, he can upload a photo of the item and its details (serial number, purchase date). At this point an employee estimates the item's price and, if < 2000$, the system sends a contract to the customers and waits for the signed copy. If the estimated price is > 2000$ the system sends a message to the customer explaining the service's conditions.

- IC 01 - Application layer

The IT system of the IC company is composed of a CRM system where information about the customers are managed to check the reliability. Moreover, an ad-hoc program called "estimator" is made available through a desktop interface to the employee to estimate the price. Finally, a web application is available to support the finalization of the contract.